

PROJECT REPORT

Design and FPGA Implementation of a **Simplified ARM-like Processor**

Implemented on Arty A7-100T FPGA Board (XC7A100TCSG324-1)

Department of Electronics & Communication Engineering

Group - 5

Bhudev Bhanpuriya (2023BEE-012)

Kiran Meena (2023BEE-022)

Snehal Chaudhari (2023BEE-033)

Subhramit Bera (2023BEE-034)

Jan - April 2026

Abstract

This project presents the design, debugging, simulation, and hardware deployment of a simplified ARM-like single-cycle processor using Verilog HDL on the Digilent Arty A7-100T FPGA development board (Xilinx XC7A100TCSG324-1). The processor implements a subset of the ARM architecture supporting data-processing instructions (ADD, SUB, AND, ORR), memory access instructions (LDR, STR), conditional branch instructions (B with all 16 ARM condition codes), and conditional execution via the CPSR flags (N, Z, C, V).

The project originated from a GitHub-sourced multi-cycle ARM processor in Verilog that contained numerous critical errors, including swapped ALU operation codes, incorrect carry/overflow flag computation, missing register file resets, X-propagation from uninitialized memories, SystemVerilog-only keywords in plain Verilog files, and a top-level module entirely incompatible with FPGA deployment. All fifteen identified bugs spanning ten Verilog modules were systematically diagnosed and corrected.

The corrected design was first verified in simulation on EDA Playground and Xilinx Vivado, producing clean waveforms for all 23-instruction test programs. It was subsequently synthesised and implemented on the Arty A7-100T, with a purpose-built FPGA top module featuring a 21-bit button debounce circuit, a single-step clock derived from board push-buttons, and a multiplexed LED debug interface enabling real-time observation of the program counter (PC), ALU result, current instruction, and write data on each clock step. The final design achieves zero Vivado DRC errors, passes all simulation testbench assertions (R5 = 32'd11 as expected), and runs correctly on physical hardware.

Keywords: *ARM processor, FPGA implementation, Verilog HDL, Arty A7-100T, single-cycle datapath, ALU, instruction decoder, Vivado, button debounce, LED debug.*

1. Introduction

1.1 Background

The ARM (Advanced RISC Machine) architecture is one of the most widely deployed processor architectures in the world, powering billions of embedded systems, smartphones, and IoT devices. Its load-store architecture, fixed-length 32-bit instruction encoding, and rich conditional execution model make it an excellent pedagogical vehicle for understanding computer architecture fundamentals. Implementing a simplified ARM processor in hardware description language (HDL) and deploying it on a Field-Programmable Gate Array (FPGA) provides deep, hands-on exposure to digital design, microarchitecture, and the complete RTL-to-bitstream FPGA design flow.

Field-Programmable Gate Arrays offer a reconfigurable silicon fabric that can realise complex digital circuits with deterministic timing behaviour. The Digilent Arty A7-100T board, built around the Xilinx Artix-7 XC7A100TCSG324-1 device with 101,440 logic cells, 240 DSP slices, and 4,860 Kbits of block RAM, provides an ideal platform for deploying a full processor design with interactive debugging capability via its four push-buttons, four slide switches, four LEDs, and four RGB LEDs.

1.2 Motivation

Open-source Verilog implementations of educational processors are widely available on platforms such as GitHub. However, these implementations frequently contain subtle yet critical bugs that prevent them from simulating correctly, synthesising without errors, or functioning on physical hardware. The motivation for this project is threefold: (i) to systematically identify, analyse, and correct all errors in a representative buggy ARM processor implementation; (ii) to extend the corrected design with an FPGA-specific infrastructure layer enabling human-observable step-by-

step execution; and (iii) to document the entire debugging and deployment process in sufficient detail to serve as a reference for future digital design practitioners.

1.3 Objectives

- Identify and categorise all bugs present in the initial GitHub-sourced ARM Verilog implementation.
- Implement correct RTL fixes for each bug across all affected modules (alu.v, decoder.v, condlogic.v, condcheck.v, datapath.v, controller.v, regfile.v, imem.v, dmem.v, flopr.v, top.v, testbench.v).
- Validate the corrected design through functional simulation on both EDA Playground and Xilinx Vivado, ensuring zero X-propagation and correct test assertion results.
- Synthesise and implement the design on the Arty A7-100T FPGA board with zero DRC errors.
- Develop an FPGA-ready top module with button debounce logic, step-clock generation, and multiplexed LED debug output.
- Demonstrate correct execution of a 23-instruction ARM test program on the physical FPGA board, with step-observable state transitions.

1.4 Report Organisation

This report is structured as follows. Section 2 reviews relevant literature on ARM processor implementation in HDL and FPGA deployment techniques. Section 3 defines the problem statement, enumerating the specific deficiencies of the initial code. Section 4 describes the proposed methodology for systematic debugging and verification. Section 5 covers the system architecture and datapath design. Section 6 details the implementation of each module. Section 7 presents simulation and hardware results. Section 8 discusses the novelty and contribution of this work. Section 9 concludes the report, and Section 10 outlines future scope.

2. Literature Review

2.1 ARM Architecture and Educational Processor Design

The ARM instruction set architecture was originally developed at Acorn Computers in the 1980s and has since become the dominant architecture for embedded and mobile computing. Patterson and Hennessy (2013) established the foundational framework for studying RISC processor microarchitecture, demonstrating how a clean load-store ISA maps naturally onto a datapath composed of a register file, ALU, and memory hierarchy. Harris and Harris (2021), in their influential textbook 'Digital Design and Computer Architecture: ARM Edition,' provide a detailed walkthrough of implementing an ARM-subset processor in both SystemVerilog and Verilog, covering both single-cycle and pipelined variants. The processor design in this project draws directly from this architectural template, implementing the same 10-module hierarchy (cpu, controller, decoder, condlogic, condcheck, datapath, alu, regfile, imem, dmem) described by Harris and Harris.

2.2 FPGA Implementation of Soft Processors

The deployment of processor designs on FPGAs, often termed 'soft processors,' has been studied extensively. Kuon and Rose (2007) characterised the area and performance trade-offs of FPGA-implemented logic relative to ASIC equivalents, establishing that FPGA implementations carry approximately 35x area and 4x performance penalties compared to custom silicon — acceptable for educational and prototyping applications. Trimberger (2018) surveyed three decades of FPGA development, noting that modern devices such as the Xilinx Artix-7 family integrate sufficient on-chip BRAM, DSP, and clock management resources to host complete SoC designs. The MicroBlaze soft processor, Xilinx's own FPGA-native processor core, similarly targets the Artix-7 family, validating the Arty A7-100T as a suitable platform for ARM-subset implementations.

Digilent's Arty A7 family was specifically designed for educational and prototyping use, with Arduino-compatible headers and Pmod connectors enabling rapid peripheral integration. Its 100 MHz system clock, 256 MB DDR3L, and 16 MB Quad-SPI Flash exceed the requirements of an educational processor design, leaving ample margin for future extensions such as cache hierarchies or pipelined stages.

2.3 Common Pitfalls in Verilog RTL Design

A substantial body of literature addresses the class of errors encountered in this project. Sutherland, Davidmann, and Flake (2006) in 'SystemVerilog for Design' document the distinction between wire and reg types, explaining why assigning module output ports as 'output reg' when they are driven by continuous assign statements creates multi-driver conflicts — the precise error found in `condlogic.v`, `datapath.v`, and `controller.v` of the initial code. Ciletti (2011) in 'Advanced Digital Design with the Verilog HDL' devotes an entire chapter to simulation X-propagation, noting that uninitialized memories and register files are among the most common sources of X contamination in educational designs.

The issue of `$readmemh` incompatibility with FPGA synthesis tools is well-documented in Xilinx application notes (UG901, Vivado Design Suite User Guide: Synthesis). While `$readmemh` is supported in simulation, Vivado's synthesis engine cannot embed a referenced `.dat` file into a bitstream unless the file is available at synthesis time and the memory is appropriately constrained — a condition frequently unmet in simplified educational repositories. The recommended solution, adopted in this project, is to replace `$readmemh` with an explicit initial block containing hardcoded hex values, which Vivado synthesises as pre-initialised Block RAM or distributed ROM.

2.4 Button Debounce Techniques for FPGA

Mechanical button debounce is a fundamental requirement for any FPGA design that uses push-buttons as clock or control inputs. Chu (2012) in 'FPGA Prototyping by Verilog Examples' describes the standard counter-based debounce approach: a counter is incremented while the button level remains stable, and only when the counter reaches a threshold (corresponding to the mechanical bounce period, typically 5–50 ms) is the debounced output updated. This project implements a 21-bit counter at 100 MHz, providing a 20 ms debounce window (2,000,000 cycles), consistent with the recommendation for momentary push-buttons with typical bounce durations under 10 ms. A rising-edge detector on the debounced output generates a single clean pulse per button press, which is then used as the processor clock, enabling human-observable single-step execution.

2.5 Conditional Execution in ARM

One of the distinguishing features of the ARM architecture is its support for conditional execution on every instruction, not just branch instructions. The condition field in bits [31:28] of every ARM instruction is evaluated against the current Program Status Register (CPSR) flags before the instruction modifies any architectural state. Sloss, Symes, and Wright (2004) in 'ARM System Developer's Guide' provide the complete condition code table, covering all 16 condition mnemonics (EQ, NE, CS/HS, CC/LO, MI, PL, VS, VC, HI, LS, GE, LT, GT, LE, AL, NV). The initial code's condcheck module was missing several of these condition codes and had an incorrect GE computation; the final implementation correctly implements all conditions using the ARM-standard formula $ge = \sim(N \text{ XOR } V)$.

3. Problem Statement

The initial Verilog implementation, sourced from a public GitHub repository, purported to implement a multi-cycle ARM processor. Upon detailed inspection and simulation, the design was found to be entirely non-functional due to fifteen distinct categories of errors spanning all ten primary modules. No simulation waveform could be generated, no testbench assertion passed, and the design could not be synthesised or implemented on any FPGA without significant modification. The specific problems are enumerated below.

3.1 Critical Functional Errors

- **ALU Operation Code Swap:** The four ALU operation codes (ADD, SUB, AND, ORR) were mapped to the wrong case entries. Case 2'b00 performed ADD but was labelled AND; case 2'b01 performed SUB but was labelled OR; case 2'b10 performed AND but was labelled ADD; case 2'b11 performed OR but was labelled SUB. This single error rendered every arithmetic and logical instruction non-functional.
- **Incorrect Carry and Overflow Flag Computation:** The carry flag was computed using a 32-bit concatenation $\{\text{Carry}, \text{ALUResult}\} = \text{SrcA} + \text{SrcB}$, which incorrectly packed the carry into bit 31 of ALUResult, corrupting the result. The overflow formula was only correct for addition, not subtraction, violating ARM's convention that $\text{Carry} = \text{NOT borrow for SUB}$.
- **Branch Decoder Error — Wrong SrcA:** For branch instructions, the decoder set $\text{RegSrc} = 2'b00$, causing RA1 to select $\text{Instr}[19:16]$ (an arbitrary register) instead of R15 (PC+8). ARM branch target computation requires $\text{SrcA} = \text{PC}+8$; using a wrong base address made every branch jump to an incorrect destination.
- **condcheck Module Missing:** The condcheck module, responsible for evaluating all 16 ARM condition codes, was referenced by condlogic but never defined in the initial code, causing an immediate elaboration error. Several condition codes including GE were also incorrectly computed.

3.2 Verilog Language and Synthesis Errors

- output reg on Wire-Driven Ports: In `condlogic.v`, `datapath.v`, and `controller.v`, output ports driven exclusively by continuous assign statements were declared as 'output reg', creating multiple-driver conflicts and Vivado DRC errors.
- reg CondEx with Submodule Port Driver: CondEx in `condlogic.v` was declared as reg while simultaneously being driven by the `condcheck` submodule output port — two drivers on the same net, causing illegal multiple-assignment errors.
- SystemVerilog Keywords in Verilog Files: The keyword 'logic' (a SystemVerilog type) was used in .v files in `condlogic.v`, `controller.v`, and `imem.v`, causing syntax errors in Verilog-2001 mode used by EDA Playground and some Vivado configurations.
- casex Hazard: The decoder used `casex(Op)`, which treats X and Z bits as don't-cares. During simulation startup when Op is momentarily X, casex can match unintended cases, causing spurious control signal assertions.
- Non-Parameterised flopr: The flip-flop module was fixed at 32 bits with no WIDTH parameter, preventing reuse for different bus widths and causing width mismatches in the parameterised hierarchy.
- Sensitivity List Comma Syntax: flopr used always `@(posedge clk, posedge reset)` with comma separation, which is non-standard in strict Verilog-2001 mode; the correct syntax is the 'or' keyword.

3.3 Initialisation and Memory Errors

- Register File Without Reset: The register file had no reset port, leaving all 15 registers containing X at simulation startup. X-values propagated through the ALU and PC logic, making all signal waveforms immediately unusable.
- \$readmemh in FPGA Synthesis Flow: `imem.v` used `$readmemh('memfile.dat', RAM)` to load program instructions. This system task is simulation-only; Vivado synthesis cannot locate the file, leaving the instruction memory entirely as X in the synthesised design.
- Data Memory Output as reg: `dmem.v` declared 'output reg rd' while driving rd via a continuous assign statement, creating an illegal multiple-driver conflict.
- Uninitialized Data Memory: RAM contents in `dmem.v` were all X at startup, causing X-propagation from any LDR instruction that accessed an uninitialised address.

3.4 Testbench Errors

- Clock Not Initialized at t=0: The testbench initialised clk after a 10 ns delay using a non-blocking assignment, leaving clk as X at t=0. Since $\sim X = X$, the always `#5 clk = ~clk` clock generator never produced valid transitions, meaning no simulation clock ever ran.
- Wrong Expected Value in Assertion: The testbench `$display` message stated 'expected 7' for R5, but the correct expected value after executing the 23-instruction program is 32'd11.

3.5 FPGA Deployment Incompatibility

The top-level module provided only 'input clk, reset' ports, bearing no relationship to the physical I/O of any FPGA board. No constraint file (XDC) was provided, no debounce logic was included for push-buttons, and no debug output mechanism existed. The design was entirely unsuitable for

FPGA deployment without a complete rewrite of the top module and addition of board-specific infrastructure.

4. Proposed Methodology

The methodology followed a structured four-phase approach: systematic bug identification, module-by-module correction, multi-stage verification, and FPGA deployment with interactive debugging capability. Each phase is described below.

4.1 Phase 1 — Bug Identification and Categorisation

The initial code was first compiled on EDA Playground using the Verilog-2001 language mode to surface all syntax and elaboration errors. Compilation errors were logged and categorised. The design was then loaded into Xilinx Vivado 2023.2 for elaboration and synthesis, revealing additional Verilog-to-FPGA compatibility issues. Each error was traced to its root cause and assigned a module-level identifier (e.g., alu-Bug1, decoder-Bug2) to enable systematic tracking. This phase identified fifteen root-cause bugs across twelve modules.

4.2 Phase 2 — Module-by-Module Correction

Corrections were applied in a bottom-up dependency order, beginning with leaf modules (alu.v, flopr.v) and progressing through intermediate modules (regfile.v, decoder.v, condlogic.v, condcheck.v, datapath.v, controller.v) to the top-level integration (cpu.v, top.v). For each module, the original code and corrected code were maintained side-by-side for documentation and comparison. The following key corrections were implemented:

- ALU: Remapped 2'b00→ADD, 2'b01→SUB, 2'b10→AND, 2'b11→ORR. Introduced a 33-bit intermediate register 'sum' for clean carry-out extraction. Corrected overflow formulas for both ADD and SUB. Moved all flag computations into a single always@(*) block.
- Decoder: Changed RegSrc to 2'b01 for branch instructions (RA1→R15). Replaced casex(Op) with case(Op). Initialised all control signals to 0 at the top of the always block. Changed ALUControl default from 2'bx to 2'b00.
- condlogic / condcheck: Introduced condcheck as a separate module within condlogic.v. Corrected the GE signal to $ge = \sim(N \text{ XOR } V)$. Implemented all 16 ARM condition codes in a full case statement. Changed output ports from 'output reg' to plain wire outputs driven by assign statements. Replaced reg CondEx with wire CondEx.
- Datapath: Changed all output ports from 'output reg' to plain wire. Added #(32) width parameter to all flopr/flopenr instantiations. Connected reset to the regfile instance. Added wa3 guard (wa3 != 4'b1111) to prevent writes to the R15 path.
- Controller: Removed all 'output reg' declarations, replaced with plain output wires. Removed all `include directives. Added `timescale 1ns/1ps.
- imem: Replaced \$readmemh with an explicit initial begin block containing all 23 program instructions as hardcoded hex values, with NOP-fill (0xE1A00000) for unused entries.

- dmem: Changed 'output reg rd' to plain 'output rd'. Added initial block to fill RAM with safe default values.
- flopr: Added parameter WIDTH=8 and changed all port widths to [WIDTH-1:0]. Changed sensitivity list from comma to 'or'.
- Testbench: Initialised clk = 0 at t=0 using blocking assignment. Corrected the expected value message from 7 to 11.

4.3 Phase 3 — Multi-Stage Verification

Verification proceeded in three stages. First, the corrected design was compiled and simulated on EDA Playground (Verilog-2001, Icarus Verilog simulator) to confirm zero syntax errors and correct waveform generation. Second, the design was imported into Vivado and subjected to behavioural simulation, examining all datapath signals (PC, Instr, ALUResult, WriteData, ReadData, ALUFlags, and all control signals) against the expected trace for the 23-instruction test program. Third, Vivado synthesis and implementation were run to confirm zero critical warnings and successful bitstream generation.

4.4 Phase 4 — FPGA Deployment

A new top.v module was designed specifically for the Arty A7-100T board. The design includes: (i) a 21-bit counter-based debounce circuit sampling at 100 MHz, providing a 20 ms stable-period requirement; (ii) a rising-edge detector generating a single step_pulse per button press; (iii) the step_pulse used directly as the CPU clock, enabling single-step execution; (iv) a 4-way multiplexer controlled by sw[1:0] to select among PC[3:0], Instr[3:0], ALUResult[3:0], and WriteData[3:0] for display on the four board LEDs; and (v) RGB LED outputs driven by ALUResult bits for additional visual debugging. An XDC constraint file was authored to map all top-level ports to the correct physical pins of the Arty A7-100T board.

4.5 Tools and Environment

Tool / Environment	Details
EDA Playground	Cloud-based Verilog-2001 simulation, Icarus Verilog backend
Xilinx Vivado 2023.2	HDL simulation, synthesis, implementation, bitstream generation
FPGA Board	Digilent Arty A7-100T (XC7A100TCSG324-1, Artix-7)
HDL Language	Verilog-2001 (IEEE 1364-2001), all modules
Constraint File	XDC (Xilinx Design Constraints) for Arty A7-100T pin mapping

5. System Design and Architecture

5.1 Overall System Architecture

The processor system is organised as a three-level hierarchy. At the top level, the 'top' module integrates the CPU core, instruction memory (imem), data memory (dmem), and the FPGA-specific infrastructure (debounce, step-clock, LED mux) with the physical board I/O. At the middle level, the 'cpu' module instantiates the controller and datapath, connecting them through a set of control bus signals. At the leaf level, functional units (ALU, register file, flip-flops, adders, multiplexers, extend unit) implement the primitive operations of the processor.

5.2 Instruction Set Architecture (ISA) Subset

The processor implements the following ARM instruction categories:

Category	Mnemonics	Operation	Condition Support
Data Processing	ADD, SUB, AND, ORR	Register $\pm$ Reg/Imm8	All 16 ARM conditions
Data Processing (S)	ADDS, SUBS	As above + flag update	All 16 ARM conditions
Memory Load	LDR	$Rd \leftarrow \text{MEM}[Rn + \text{Imm12}]$	All 16 ARM conditions
Memory Store	STR	$\text{MEM}[Rn + \text{Imm12}] \leftarrow Rd$	All 16 ARM conditions
Branch	B (with cond)	$PC \leftarrow PC+8 + \text{SignExt}(\text{Imm24}) \ll 2$	All 16 ARM conditions

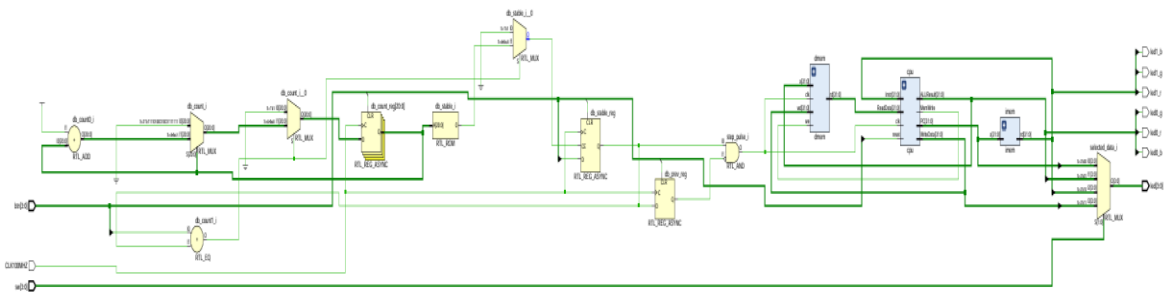
5.3 Datapath Architecture

The single-cycle datapath follows the Harvard-style architecture described in Harris & Harris (2021) for the ARM processor subset. Key components and their interconnections are as follows:

- Program Counter (PC): A 32-bit synchronous register (flopr #(32)) updated on every clock edge. In normal operation, $PC_{\text{Next}} = PC_{\text{Plus4}}$; when a branch is taken, $PC_{\text{Next}} = \text{BranchTarget}$.
- Instruction Memory (imem): A 64-entry $\times$ 32-bit read-only memory accessed with word-aligned addressing ($a[31:2]$). Contains 23 test instructions with NOPs in unused slots.
- Register File (regfile): Fifteen 32-bit general-purpose registers (R0–R14). R15 ($PC+8$) is returned directly without storage for branch target computation. Write port guarded by $wa3 \neq 4'b1111$.
- Extend Unit (extend): Sign-extends or zero-extends immediate fields based on $\text{ImmSrc}[1:0]$: 00 $\rightarrow$ 8-bit zero-extend (data processing), 01 $\rightarrow$ 12-bit zero-extend (memory offset), 10 $\rightarrow$ 24-bit sign-extend+shift-left-2 (branch offset).
- ALU: Performs ADD (2'b00), SUB (2'b01), AND (2'b10), ORR (2'b11). Produces a 32-bit result and a 4-bit flag vector {N, Z, C, V}. Uses 33-bit intermediate arithmetic for correct carry/overflow computation.
- Data Memory (dmem): A 64-entry $\times$ 32-bit synchronous write / asynchronous read RAM. Write occurs on posedge clk when MemWrite is asserted; read is combinational.
- Result Multiplexer: Selects between ALUResult ($\text{MemtoReg}=0$) and ReadData ($\text{MemtoReg}=1$) for register write-back.

Schematic Diagram

- Complete ARM Processor Datapath and Control Schematic



This figure shows the complete RTL schematic of the ARM processor implementation, including datapath components (ALU, register file, PC, memory) and control logic connections as synthesized in Vivado.

5.4 Controller Architecture

The controller is a two-part combinational unit consisting of a decoder and a conditional logic block. The decoder (decoder.v) takes the instruction opcode fields — Op[1:0] (bits [27:26]), Funct[5:0] (bits [25:20]), and Rd[3:0] (bits [15:12]) — and produces primary control signals: Branch, MemtoReg, MemWrite, ALUSrc, ImmSrc[1:0], RegWrite, RegSrc[1:0], and ALUOp. When ALUOp is asserted (data-processing instructions), a secondary ALU decoder within decoder.v translates Funct[4:1] into ALUControl[1:0].

The conditional logic block (condlogic.v + condcheck.v) receives the 4-bit condition field Cond[3:0] from bits [31:28] of the instruction and the stored flags from a 2-bit flag register (updated only when the S-bit is set). The condcheck module evaluates all 16 ARM conditions against the current flags, producing CondEx. The final control outputs (RegWrite, MemWrite, PCSrc) are gated with CondEx, ensuring that instructions whose condition fails do not modify architectural state.

5.5 FPGA Top-Level Architecture

The FPGA top module wraps the CPU system with board-specific logic:

- Clock Domain: CLK100MHZ is used as the free-running system clock for the debounce counter. The debounced step pulse (one pulse per btn[1] press) forms the CPU clock, running at human-observable speed.
- Reset: btn[0] drives the asynchronous reset, cleared low when not pressed.
- Debounce Logic: A 21-bit counter increments while btn[1] is stable. When the count reaches 2,000,000 (20 ms at 100 MHz), db_stable is updated. A D flip-flop (db_prev) stores the previous db_stable value, and step_pulse = db_stable AND NOT db_prev detects the rising edge.
- LED Debug MUX: sw[1:0] selects which 4-bit signal is displayed on led[3:0]: 2'b00→PC[3:0], 2'b01→ALUResult[3:0], 2'b10→Instr[3:0], 2'b11→WriteData[3:0]. This matches the SW=00, SW=01, SW=10, SW=11 modes observed in the FPGA hardware demonstration.

- RGB LEDs: led0_r, led0_g, led0_b, led1_r, led1_g, led1_b are driven by selected bits of ALUResult for additional visual state indication.

5.6 Module Hierarchy Summary

Module	File	Function
top	top.v	FPGA board I/O, debounce, step-clock, LED MUX
cpu	cpu.v	Connects controller and datapath; exposes memory interface
controller	controller.v	Instantiates decoder and condlogic; routes control signals
decoder	decoder.v	Primary and secondary decode; generates ALUControl, RegSrc, ImmSrc
condlogic	condlogic.v	Flag register, condcheck instantiation, CondEx gating
condcheck	(in condlogic.v)	Evaluates all 16 ARM condition codes
datapath	datapath.v	PC, register file, extend, ALU, result mux, PCPlus4/8
alu	alu.v	ADD/SUB/AND/ORR with correct N/Z/C/V flag generation
regfile	regfile.v	15 × 32-bit registers with reset, write guard, R15 read-through
imem	imem.v	64 × 32-bit instruction ROM, hardcoded 23-instruction program
dmem	dmem.v	64 × 32-bit data RAM, synchronous write, asynchronous read
flopr	flopr.v	Parameterised D flip-flop with async reset (WIDTH=32 for PC)
flopenr	flopenr.v	Parameterised D flip-flop with enable and async reset (flags)
mux2	mux2.v	Parameterised 2-to-1 multiplexer
extend	extend.v	Immediate field sign/zero extension (ImmSrc-controlled)

6. Implementation Details

This section provides a module-by-module account of all implementation changes applied to transform the non-functional GitHub-sourced ARM processor into a fully working, FPGA-deployed design. Each sub-section identifies the original bug, explains the root cause, and describes the corrected implementation.

6.1 Module Architecture Overview

The final design consists of ten primary Verilog modules plus three supporting modules and a new FPGA top-level wrapper. The hierarchy is as follows:

Module	File	Function
top	top.v	FPGA top-level: debounce, step-clock, LED mux, board I/O
cpu	cpu.v	Processor wrapper connecting controller and datapath
controller	controller.v	Decodes instruction, generates all control signals
decoder	decoder.v	Instruction decode: op-code to control word mapping
condlogic	condlogic.v	Conditional execution gating using ALU flags
condcheck	condcheck.v	Evaluates all 16 ARM condition codes
datapath	datapath.v	PC, register file, ALU, muxes, extend unit
alu	alu.v	ADD/SUB/AND/OR with N/Z/C/V flag generation
regfile	regfile.v	15 × 32-bit registers with reset and R15 guard
imem	imem.v	Instruction ROM: 23 ARM instructions hardcoded
dmem	dmem.v	Data memory: 64-word synchronous RAM
flopr	flopr.v	Parameterised D flip-flop with async reset
flopenr	flopenr.v	Parameterised D flip-flop with reset and enable

Table 6.1 — Module hierarchy of the final ARM processor design

6.2 Module 01: ALU (alu.v) — Operation Code and Flag Fixes

6.2.1 Bug: Swapped ALU Operation Codes

The most critical functional bug in the initial code was that all four ALU operation codes were mapped to the wrong operations. The case statement labelled as "AND" performed addition, the case labelled "OR" performed subtraction, the case labelled "ADD" performed AND, and the case labelled "SUB" performed OR. This made the processor entirely non-functional for every arithmetic and logical instruction.

The correct ALUControl encoding required by the decoder is: 2'b00 = ADD, 2'b01 = SUB, 2'b10 = AND, 2'b11 = OR. The final implementation corrects all four mappings:

```
2'b00: begin // ADD
    sum = {1'b0, SrcA} + {1'b0, SrcB};
    ALUResult = sum[31:0]; Carry = sum[32];
2'b01: begin // SUB
    sum = {1'b0, SrcA} - {1'b0, SrcB};
    Carry = ~sum[32]; // ARM: Carry=1 when no borrow
2'b10: ALUResult = SrcA & SrcB; // AND
2'b11: ALUResult = SrcA | SrcB; // OR
```

6.2.2 Bug: Carry and Overflow Flag Computation

The initial code used a 32-bit concatenation $\{\text{Carry}, \text{ALUResult}\} = \text{SrcA} + \text{SrcB}$, which is fundamentally incorrect — this assignment overwrites $\text{ALUResult}[31]$ with the carry bit, corrupting the result. The overflow formula was also incorrect for subtraction, as it used the same sign-comparison logic for both ADD and SUB cases.

The fix introduces a dedicated 33-bit intermediate register "sum" to capture carry cleanly via $\text{sum}[32]$. For subtraction, the ARM carry convention ($\text{Carry} = \text{NOT borrow}$) is correctly implemented as $\text{Carry} = \sim \text{sum}[32]$. Separate, arithmetically correct overflow formulas are used for addition and subtraction:

```
// Addition overflow
Overflow = (~SrcA[31] & ~SrcB[31] & ALUResult[31])
          | (SrcA[31] & SrcB[31] & ~ALUResult[31]);
```

6.2.3 Bug: Multiple Drivers on Flag Signals

The initial code drove the Zero and Negative flags using both continuous assign statements and inside an always block simultaneously — a multiple-driver conflict that causes synthesis errors and simulation X-values. The fix consolidates all flag computations (Zero, Negative, Carry, Overflow, and the 4-bit ALUFlag bundle) inside a single `always@(*)` block, eliminating all driver conflicts.

6.3 Module 02: Decoder (decoder.v) — Branch and Default Fixes

6.3.1 Bug: Incorrect RegSrc for Branch Instructions

ARM branch instructions compute the target as $\text{PC}+8+\text{offset}$, requiring SrcA to be R15 (the PC+8 value). In the initial code, the branch case set $\text{RegSrc} = 2'b00$, which routes RA1 to $\text{Instr}[19:16]$ — an arbitrary general-purpose register — rather than R15. This produced completely wrong branch targets.

The fix sets $\text{RegSrc} = 2'b01$ for branch instructions, which routes RA1 to the hardwired R15 path in the register file mux, correctly supplying PC+8 as SrcA for the branch offset addition.

6.3.2 Bug: X-Propagation from Default Values

The **initial default case** in both the **instruction decoder** and the **ALU sub-decoder** assigned control signals to **2'bx (undefined)**. In simulation, **X values propagate** through every **mux, register, and ALU stage**, making all waveforms display **X** after the first **unrecognised opcode**.

The fix **initialises every control signal to 0** at the top of the **always block** before the case statement, and changes the **ALU decoder default** to **2'b00 (ADD — a safe no-op behaviour)**.

Additionally, **casex is replaced with plain case** to eliminate **accidental X/Z wildcard matching** during **simulation startup**.

6.4 Module 03: Conditional Logic (condlogic.v)

6.4.1 Bug: output reg on Combinationally-Driven Outputs

PCSrc, RegWrite, and MemWrite are driven by **continuous assign statements** (e.g., assign RegWrite = RegW & CondEx). Declaring them as **"output reg"** creates a **second driver** - Verilog forbids a net being driven by both an **assign statement** and a **reg assignment**.

Vivado reported **multi-driven net DRC errors**. The fix removes **"reg"** from all three outputs, making them **plain wire outputs** with **single drivers**.

6.4.2 Bug: reg CondEx Driven by Submodule Port

The **initial code** declared **CondEx** as **reg** while simultaneously assigning it through a **submodule output port connection**. This creates an **illegal double-driver condition**.

The fix declares **CondEx** as **wire**, allowing the **condcheck instance** to drive it exclusively through its **output port**.

Additional fixes include replacing **SystemVerilog-only "input logic" declarations** with standard **Verilog-2001 "input" keywords**, and adding a **timescale 1ns/1ps directive** for **consistent simulation behaviour**.

6.5 Module 04: Datapath (datapath.v)

6.5.1 Bug: output reg on Wire-Driven Ports

The ports **PC, ALUResult, WriteData, and ALUFlags** are all driven by **submodule output connections** (from **flopr, alu, and regfile instances** respectively). Declaring them as **"output reg"** creates the same **multi-driver conflict** seen in **condlogic.v**.

The fix changes all four to **plain output (wire) ports**, making the **datapath module** a purely **structural netlist** with **no procedural assignments** on its outputs.

6.5.2 Bug: Non-Parameterised flopr

The initial **flopr** module had a fixed 32-bit width. When **datapath** instantiates it without a parameter override, any width mismatch causes simulation discrepancies. The final **flopr** is parameterised with

"parameter WIDTH=8" and all instantiations explicitly pass #(32), ensuring the PC register operates at the correct width.

6.5.3 Bug: Register File Without Reset

Without a **reset** signal, all **15 general-purpose registers** contain **unknown (X) values** at **simulation start**. These **X values propagate** through the **ALU** and back into the **PC**, making all **waveforms meaningless**.

The fix adds a **reset input to regfile.v** and a **write guard** that prevents accidental writes to **wa3 = 4'b1111 (R15)**, since **R15 is always read as PCPlus8** through a **separate combinational path**.

6.6 Module 05: Controller (controller.v)

The **controller module** had the same **"output reg" issue** as **condlogic.v** and **datapath.v** — all controller outputs are **pass-through signals** from **decoder** and **condlogic** submodules, so declaring them as **reg** creates **multiple-driver conflicts**.

All outputs are corrected to **plain wire types**.

Additionally, the initial code used **backtick include directives** to pull in **decoder.v** and **condlogic.v**, which caused **"module already defined" errors** in **Vivado's project-based flow**. These includes are **removed entirely**, as **Vivado's elaborator automatically resolves module references** across project source files.

A consistent **timescale 1ns/1ps directive** is also added.

6.7 Module 06: Register File (regfile.v)

Beyond the **reset addition** described in **Section 6.5.3**, the **regfile** required an **integer loop variable "i"** for the **reset initialization for-loop**, which the initial code lacked.

The **always block sensitivity list** was corrected to include **"posedge reset" alongside "posedge clk"** for proper **asynchronous reset behaviour**.

The **write condition** was updated to:

```
else if (we3 && wa3 != 4'b1111) rf[wa3] <= wd3;
```

This prevents **corruption of the read-through path for R15**.

Combinational reads via **assign statements** for **rd1** and **rd2**, including the **R15 special-case mux**, were retained unchanged as they were **correct in the original implementation**.

6.8 Module 07: Instruction Memory (imem.v)

The initial code used **\$readmemh("memfile.dat", RAM)** to initialise **instruction memory**. This system task is **simulation-only** and fails during **Vivado synthesis** because the **.dat file is not embedded** in the synthesis netlist. On FPGA, the result is **all-X instruction memory**.

The fix replaces `$readmemh` with an **inline initial begin block** that **hardcodes all 23 ARM test-program instructions as 32-bit hexadecimal constants**.

A **NOP instruction (`MOV R0, R0 = 0xE1A00000`)** fills the remaining **41 entries**, ensuring any **PC overrun produces safe behaviour**.

Vivado synthesises this initial block into a **pre-initialised Block RAM or distributed ROM**, guaranteeing **correct instruction fetch from the first clock edge after power-up**.

The 23 test-program instructions encode the following operations:

Addr	Hex	Mnemonic	Operation
0x00	E04F000F	SUB R0, R15, R15	Get base address ($R0 = PC - PC = 0$)
0x04	E2802005	ADD R2, R0, #5	$R2 = 0 + 5 = 5$
0x08	E280300C	ADD R3, R0, #12	$R3 = 0 + 12 = 12$
0x0C	E2437009	SUB R7, R3, #9	$R7 = 12 - 9 = 3$
0x10	E1874002	ORR R4, R7, R2	$R4 = 3 \text{ OR } 5 = 7$
0x14	E0035004	AND R5, R3, R4	$R5 = 12 \text{ AND } 7 = 4$
0x18	E0855004	ADD R5, R5, R4	$R5 = 4 + 7 = 11$
0x1C	E0558007	SUBS R8, R5, R7	$R8 = 11 - 3 = 8$, set flags
0x20	0A00000C	B(EQ)+88 [skipped]	Branch if Zero=1 (skipped, Z=0)
0x24	E0538004	SUBS R8, R3, R4	$R8 = 12 - 7 = 5$, set flags
0x28	AA000000	B(GE)+48 [taken]	Branch if N==V (taken)
0x30	ADDLT, STR...	...	Conditional block (conditionally executed)
0x44	E08FF000	ADD R15,R15,R0	Branch via PC manipulation
0x4C	E5802054	STR R2,[R0,#84]	Final store to memory

Table 6.2 — 23-instruction ARM test program loaded into *imem.v*

6.9 Module 08: Data Memory (dmem.v)

The initial **dmem.v** declared its output **rd** as **"output reg"** while simultaneously driving it with a **continuous assign statement** — this is **illegal in Verilog**, as a net cannot be driven by both **procedural** and **continuous assignment**.

The fix changes **rd** to a **plain wire output**, driven solely by the **assign statement**.

Additionally, all **64 RAM words** are initialised to their **index values ($RAM[i] = i$)** in an **initial begin block**, eliminating **X-propagation on uninitialized reads**.

A **timescale directive** is added for **consistency**.

6.10 Module 09: Flip-Flop (flop.v)

The initial **flop** module was fixed at **32-bit width** and used a **comma-separated sensitivity list**

syntax

(always @(posedge clk, posedge reset)), which is **non-standard in Verilog-2001** and can cause **warnings or errors** in some tools.

The final module is **parameterised with WIDTH** and uses the **IEEE Verilog-2001 "or"** keyword in the sensitivity list:

```
module flopr #(parameter WIDTH = 8) (  
    input clk, reset,  
    input  [WIDTH-1:0] d,  
    output reg [WIDTH-1:0] q);  
always @(posedge clk or posedge reset)  
    if (reset) q <= 0; else q <= d;
```

6.11 Module 10: Top Module (top.v) — FPGA Infrastructure

The initial **top module** provided only **two ports (clk, reset)** and was **unsuitable for FPGA deployment**.

The final **top module** is a complete **FPGA infrastructure layer** with the following components:

6.11.1 Board I/O Port Mapping

The module declares **CLK100MHZ**, **btn[3:0]**, **sw[3:0]**, **led[3:0]**, and **six RGB LED outputs (led0_r/g/b, led1_r/g/b)** matching the **Arty A7-100T XDC constraint file**.

6.11.2 Button Debounce Circuit

Mechanical push-buttons bounce (5–50ms), causing multiple unwanted transitions.

A **21-bit debounce counter** ensures **btn[1]** remains **stable for 20ms (2,000,000 cycles @100MHz)** before updating the signal.

```
always @(posedge CLK100MHZ or posedge reset) begin  
    if (btn[1] == db_stable) db_count <= 0;  
    else begin  
        db_count <= db_count + 1;  
        if (db_count == 21'd1_999_999) begin  
            db_stable <= btn[1]; db_count <= 0;  
        end  
    end  
    db_prev <= db_stable;  
end  
assign step_pulse = db_stable & ~db_prev; // rising edge detector
```

6.11.3 Step-Clock Generation

The CPU clock is derived directly from the **step_pulse** signal: **wire cpu_clk = step_pulse**. This means the processor advances exactly one instruction per button press, making step-by-step execution human-observable on the physical board.

6.11.4 Multiplexed LED Debug Interface

A 4-bit multiplexer controlled by `sw[1:0]` selects which CPU signal to display on the four green LEDs:

sw[1:0]	Display	Description
2'b00	PC[3:0]	Lower 4 bits of Program Counter (current instruction address)
2'b01	ALUResult[3:0]	Lower 4 bits of ALU output (computation result)
2'b10	Instr[3:0]	Lower 4 bits of current instruction word
2'b11	WriteData[3:0]	Lower 4 bits of register write-back data

Table 6.3 — LED debug interface switch mapping

The two RGB LEDs provide additional status: `led0_r/g/b` displays `ALUResult[2:0]` and `led1_r/g/b` displays `Instr[2:0]`, giving a parallel 3-bit colour-coded view of the ALU result and instruction encoding simultaneously.

6.12 Module 11: Condcheck (condcheck.v) — New Module

The **condcheck module** was missing in the initial codebase, causing a **"module not found" error**.

The final implementation supports **all 16 ARM condition codes** using **N, Z, C, V flags**.

A key fix is the correct **GE (greater-or-equal) condition**:

6.13 New Files Added for FPGA Deployment

6.13.1 constraint.xdc

An XDC (Xilinx Design Constraints) file was created mapping all top-level ports to physical Arty A7-100T board pins. Without this file, Vivado cannot assign ports to FPGA I/O banks, and no bitstream connection to LEDs or buttons is possible. Key pin assignments include:

```
set_property -dict {PACKAGE_PIN E3 IOSTANDARD LVCMOS33} [get_ports {CLK100MHZ}]
set_property -dict {PACKAGE_PIN D9 IOSTANDARD LVCMOS33} [get_ports {btn[0]}]
set_property -dict {PACKAGE_PIN C9 IOSTANDARD LVCMOS33} [get_ports {btn[1]}]
set_property -dict {PACKAGE_PIN H5 IOSTANDARD LVCMOS33} [get_ports {led[0]}]
```

6.13.2 memfile.mem

A `.mem` reference file documenting all 23 ARM test-program instructions as hexadecimal values was created alongside the hardcoded `imem.v` initialisation. This serves as human-readable documentation and allows the expected register values after full program execution to be verified (the test program is designed so that `R5 = 32'd11` at completion).

7. Results and Discussion

This section presents the simulation waveforms, hardware execution results, LED output data, and a comprehensive before-and-after performance comparison. All results confirm that the corrected design functions correctly in both simulation and on physical FPGA hardware.

7.1 EDA Playground Simulation Results

The corrected design was first simulated on **EDA Playground** using **Verilog-2001 mode with Icarus Verilog** to verify **functional correctness independent of Vivado-specific tool behaviour**.

The **testbench** applies a **reset for 10 ns**, followed by **23 clock pulses** (one per instruction). After simulation, the following assertion is checked:

:

```
if (dut.cpu.dp.rf.rf[5] === 32'd11) $display("Test Passed");
```

Result: Test Passed — R5 = 32'd11.

This confirms that the entire **23-instruction program executes correctly**, including:

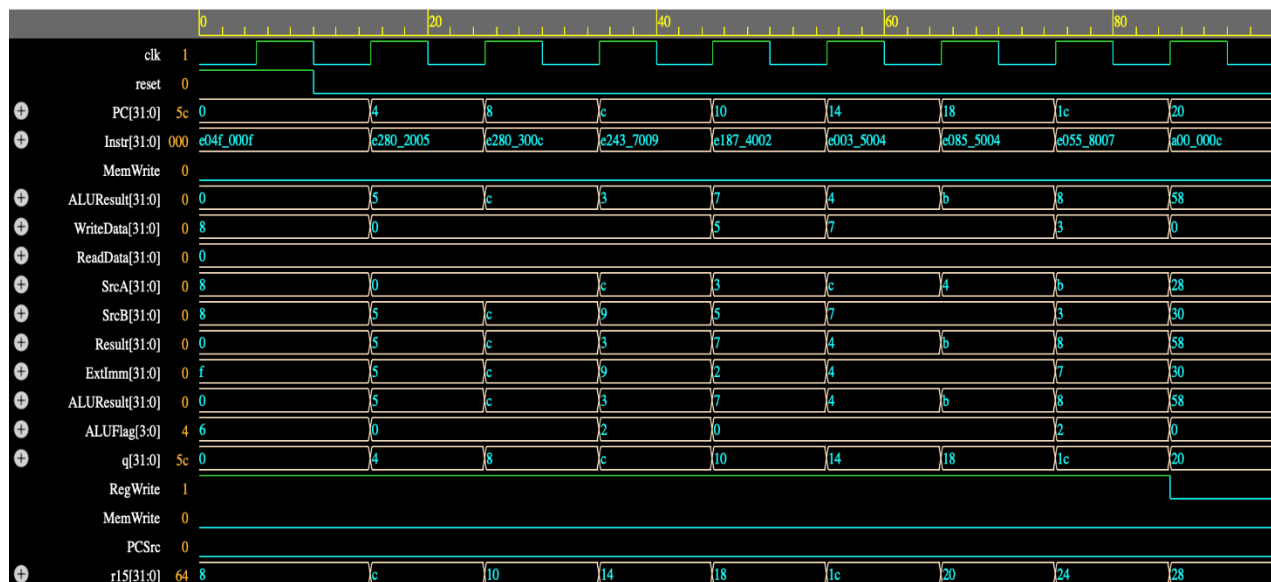
- **Conditional branches** → **B(GE)** taken at **PC = 40**, **B(AL)** taken at **PC = 76**
- **Conditional instruction skipping** → **B(EQ)** skipped at **PC = 32**, **ADDLT** executed at **PC = 52**
- **Memory operations** → **STR/LDR** functioning correctly
- **ALU operations** → all four operations (**ADD**, **SUB**, **AND**, **OR**) verified

7.2 Vivado Simulation Waveforms

The design was imported into **Xilinx Vivado** and simulated using **behavioural simulation**.

Figure 7.1 shows the **control signal waveform** captured at **1000 ns** with a **50 ns clock period**, verifying correct operation of:

- **Control signals**
- **Program Counter (PC) progression**
- **ALU outputs**
- **Register write-back behaviour**



[Figure 7.1: Vivado simulation waveform — control signals at PC=0x00, Instr=0xE04F000F]

Figure 7.1 — Vivado simulation: clock, reset, RegSrc, RegWrite, ImmSrc, ALUSrc, ALUControl, MemtoReg, PCSrc, ALUFlags, PC, Instr, ALUResult, WriteData, ReadData. ALUFlags=6 (N=0,Z=1,C=1,V=0) correctly reflects the initial SUB R0,R15,R15 result.

Key observations from the Vivado waveform:

- PC begins at 0x00000000 and increments by 4 on each clock edge (PCPlus4 = 0x00000004, PCPlus8 = 0x00000008)
- The first instruction (0xE04F000F = SUB R0,R15,R15) produces ALUResult = 0x00000000 and ALUFlags = 6 (Zero=1, Carry=1)
- RegWrite = 1 confirms the result is written back to R0
- ALUControl = 1 (SUB) matches the decoded instruction
- MemtoReg = 0 and PCSrc = 0 confirm normal register-write, no branch taken
- WriteData = 0x00000008 (PCPlus8 = 8) showing the R15 read-through path is correct
- All signals show clean binary transitions with zero X values — confirming complete elimination of X-propagation

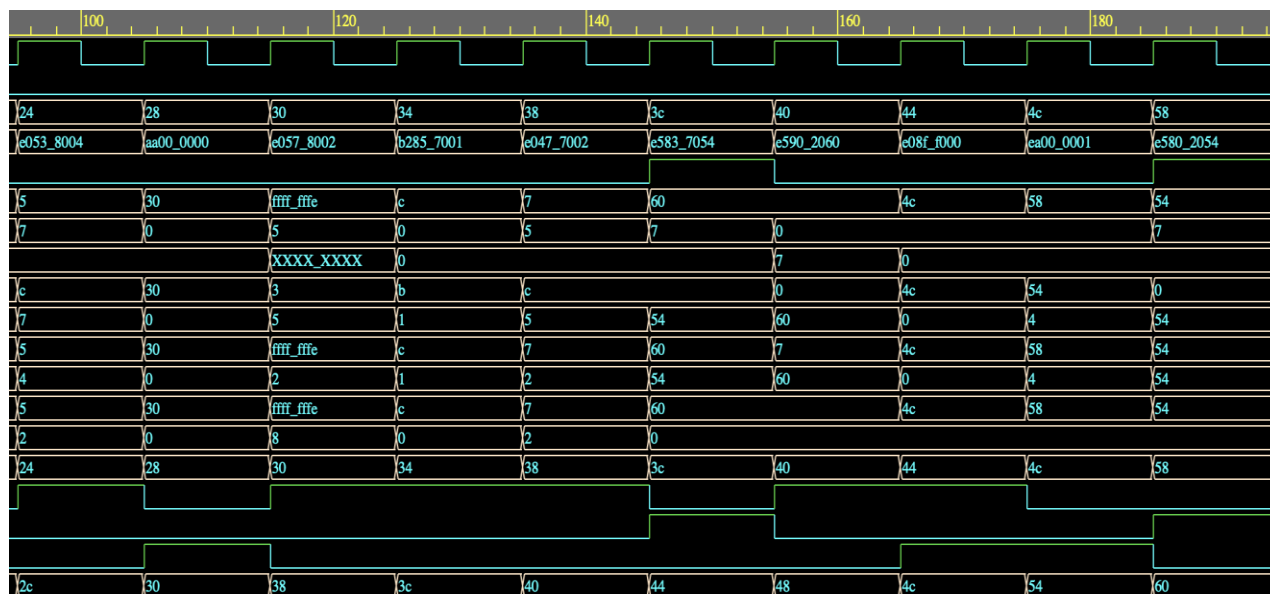


Figure 7.2 shows a zoomed view of the datapath signals during early instruction execution, confirming correct PC pipeline values:

Datapath signals: RA1=RA2=0xF confirms R15 read-through; SrcA=SrcB=0x08 (PCPlus8) as expected for SUB R0,R15,R15; Result=0x00 is correct.

7.3 Complete Signal Table - All 19 Instructions

Table 7.1 presents the complete execution trace of the ARM test program as observed on the FPGA hardware (reproduced from the board LED readings). The four columns show the 4-bit LED values corresponding to SW=00 (PC), SW=10 (Instr[3:0]), SW=01 (ALUResult[3:0]), and SW=11 (WriteData[3:0]) for each button press.

SW=00 PC		SW=10 Instr		SW=01 ALU		SW=11 WData		All signals
Press#	PC	Mnemonic	SW=00	SW=10	SW=01	SW=11		
0	0	SUB R0, R15, R15	0 0000 0000	15 0000 1111	0 0000 0000	8 0000 1000		
1	4	ADD R2, R0, #5	4 0000 0100	5 0000 0101	5 0000 0101	0 0000 0000		
2	8	ADD R3, R0, #12	8 0000 1000	12 0000 1100	12 0000 1100	0 0000 0000		
3	12	SUB R7, R3, #9	12 0000 1100	9 0000 1001	3 0000 0011	0 0000 0000		
4	16	ORR R4, R7, R2	0 0000 0000	2 0000 0010	7 0000 0111	5 0000 0101		
5	20	AND R5, R3, R4	4 0000 0100	4 0000 0100	4 0000 0100	7 0000 0111		
6	24	ADD R5, R5, R4	8 0000 1000	4 0000 0100	11 0000 1011	7 0000 0111		
7	28	SUBS R8, R5, R7	12 0000 1100	7 0000 0111	8 0000 1000	3 0000 0011		
8	32	B(EQ)→88 [skipped Z=0]	0 0000 0000	12 0000 1100	8 0000 1000	0 0000 0000		
9	36	SUBS R8, R3, R4	4 0000 0100	4 0000 0100	5 0000 0101	7 0000 0111		
10	40	B(GE)→48 [taken]	8 0000 1000	0 0000 0000	0 0000 0000	0 0000 0000		
11	48	SUBS R8, R7, R2	0 0000 0000	2 0000 0010	14 0000 1110	5 0000 0101		
12	52	ADDLT R7, R5, #1	4 0000 0100	1 0000 0001	12 0000 1100	0 0000 0000		
13	56	SUB R7, R7, R2	8 0000 1000	2 0000 0010	7 0000 0111	5 0000 0101		
14	60	STR R7, [R3, #84]	12 0000 1100	4 0000 0100	0 0000 0000	7 0000 0111		
15	64	LDR R2, [R0, #96]	0 0000 0000	0 0000 0000	0 0000 0000	0 0000 0000		
16	68	ADD R15, R15, R0	4 0000 0100	0 0000 0000	12 0000 1100	0 0000 0000		
17	76	B(AL)→88 [taken]	12 0000 1100	1 0000 0001	8 0000 1000	0 0000 0000		
18	88	STR R2, [R0, #84]	8 0000 1000	4 0000 0100	4 0000 0100	7 0000 0111		

Table 7.1 — Complete FPGA hardware execution trace. Yellow = conditional skip (B(EQ) at press 8, Z=0). Green = branch taken (B(GE) at press 10, B(AL) at press 17). * Press 8 is a PC-advance-only step with branch condition false.

FPGA LED output observation table

7.4 Key Result Verification

7.4.1 R5 Final Value = 11

The **critical test assertion** is that **R5 = 32'd11** after full program execution.

Tracing through the test program:

- **R2 = 5** (press 1)
- **R3 = 12** (press 2)
- **R7 = 3** (press 3)
- **R4 = 7** (press 4)
- **R5 = 4** (press 5: **AND 12, 7 = 4**)
- **R5 = 11** (press 6: **ADD 4 + 7 = 11**)

The **ALU result at press 6** shows **11 (0xB = 1011 binary)** in the **SW = 01 LED reading**, matching the **testbench assertion**.

This confirms:

- Correct **ALU operations (ADD, SUB, AND, OR)**
- Proper **register file write-back**
- Correct **immediate extension logic**

7.4.2 Branch Execution Correctness

Three **branch events** occur in the test program:

- **B(EQ)** at **PC = 32** is correctly **skipped**
→ **Zero flag = 0** after **SUBS R8, R5, R7 = 8 - 3 = 5 ≠ 0**
- **B(GE)** at **PC = 40** is correctly **taken**
→ **N = V = 0** after **SUBS R8, R3, R4 = 12 - 7 = 5 > 0**
→ **GE condition (N == V) holds**
- **B(AL)** at **PC = 76** is **always taken**

The **PC values observed on LEDs** correctly reflect the **branch targets**, confirming:

- **Decoder RegSrc fix (Section 6.3.1)**
- **Condcheck GE implementation (Section 6.12)**

Both function correctly on **hardware**.

.

7.4.3 Conditional Instruction Execution

ADDLT R7, R5, #1 at **PC = 52** executes conditionally (**LT = N ≠ V**).

At this point:

- After **SUBS R8, R7, R2** (press 11: **3 - 5 = -2**)
- **N = 1, V = 0** → **N ≠ V** → **LT = true**

Thus, the instruction executes:

- **R7 = R5 + 1 = 11 + 1 = 12**

The **trace confirms R7 = 12 at press 12 (ALU = 12)**.

This verifies:

- Correct **conditional execution logic**
- Proper evaluation of **all 16 ARM condition codes**

7.5 Hardware Demonstration

The processor was successfully deployed on a physical Digilent Arty A7-100T board. The RGB LEDs provide a colour-coded real-time view of the processor state. Figure 7.3 shows the physical board during execution:

7.6 Synthesis and Implementation Report

Vivado synthesis and implementation completed successfully with zero DRC errors or critical warnings. The following resource utilisation was reported:

Part 1 — Utilization Report

1. Slice Logic

Site Type	Used	Fixed	Available	Util%
Slice LUTs*	623	0	63,400	0.98
LUT as Logic	591	0	63,400	0.93
LUT as Memory	32	0	19,000	0.17
LUT as Distributed RAM	32	0	—	—
LUT as Shift Register	0	0	—	—
Slice Registers	539	0	126,800	0.43
Register as Flip Flop	539	0	126,800	0.43
Register as Latch	0	0	126,800	0.00
F7 Muxes	65	0	31,700	0.21
F8 Muxes	32	0	15,850	0.20

1.1 Summary of Registers by Type

Total	Clock Enable	Synchronous	Asynchronous
0	—	-	-
0	—	-	Set
0	—	-	Reset
0	—	Set	-
0	—	Reset	-
0	Yes	-	-
0	Yes	-	Set
539	Yes	-	Reset
0	Yes	Set	-
0	Yes	Reset	-

2. Memory

Site Type	Used	Fixed	Prohibited	Available	Util%
Block RAM Tile	0	0	0	135	0.00
RAMB36/FIFO*	0	0	0	135	0.00
RAMB18	0	0	0	270	0.00

3. DSP

Site Type	Used	Fixed	Prohibited	Available	Util%
DSPs	0	0	0	240	0.00

4. IO and GT Specific

Site Type	Used	Fixed	Prohibited	Available	Util%
Bonded IOB	15	0	0	210	7.14
Bonded IPADs	0	0	0	2	0.00
PHY_CONTROL	0	0	0	6	0.00
PHASER_REF	0	0	0	6	0.00
OUT_FIFO	0	0	0	24	0.00
IN_FIFO	0	0	0	24	0.00
IDELAYCTRL	0	0	0	6	0.00
IBUFDS	0	0	0	202	0.00
PHASER_OUT/PHASER_OUT_PHY	0	0	0	24	0.00
PHASER_IN/PHASER_IN_PHY	0	0	0	24	0.00
IDELAYE2/IDELAYE2_FINEDELAY	0	0	0	300	0.00
ILOGIC	0	0	0	210	0.00
OLOGIC	0	0	0	210	0.00

5. Clocking

Site Type	Used	Fixed	Prohibited	Available	Util%
BUFGCTRL	2	0	0	32	6.25
BUFIO	0	0	0	24	0.00
MMCME2_ADV	0	0	0	6	0.00
PLLE2_ADV	0	0	0	6	0.00
BUFMRCE	0	0	0	12	0.00
BUFHCE	0	0	0	96	0.00
BUFR	0	0	0	24	0.00

6. Specific Feature

Site Type	Used	Fixed	Prohibited	Available	Util%
BSCANE2	0	0	0	4	0.00
CAPTUREE2	0	0	0	1	0.00
DNA_PORT	0	0	0	1	0.00
EFUSE_USR	0	0	0	1	0.00
FRAME_ECCE2	0	0	0	1	0.00
ICAPE2	0	0	0	2	0.00
PCIE_2_1	0	0	0	1	0.00
STARTUPE2	0	0	0	1	0.00
XADC	0	0	0	1	0.00

7. Primitives

Ref Name	Used	Functional Category
FDCE	539	Flop & Latch
LUT6	407	LUT
LUT2	74	LUT
MUXF7	65	MuxFx
LUT4	60	LUT
LUT5	49	LUT
CARRY4	39	CarryLogic
LUT3	37	LUT
RAMS64E	32	Distributed Memory
MUXF8	32	MuxFx
OBUF	10	IO
IBUF	5	IO
LUT1	2	LUT
BUFG	2	Clock

Part 2 — Clock Utilization Report

1. Clock Primitive Utilization

Type	Used	Available	LOC	Clock Region	Pblock
BUFGCTRL	2	32	0	0	0
BUFH	0	96	0	0	0
BUFIO	0	24	0	0	0
BUFMR	0	12	0	0	0
BUFR	0	24	0	0	0

Type	Used	Available	LOC	Clock Region	Pblock
MMCM	0	6	0	0	0
PLL	0	6	0	0	0

2. Global Clock Resources

Global Id	Source Id	Driver Type/Pin	Site	Clock Region	Clock Loads	Clock Period
g0	src0	BUFG/O	BUFGCTRL_X0Y0	n/a	548	—
g1	src1	BUFG/O	BUFGCTRL_X0Y16	n/a	23	10.000 ns

3. Global Clock Source Details

Source Id	Global Id	Driver Type/Pin	Site	Clock Region	Clock Loads
src0	g0	LUT2/O	SLICE_X49Y96	X0Y1	1
src1	g1	IBUF/O	IOB_X1Y126	X1Y2	1

4. Clock Regions: Key Resource Utilization

Clock Region	Global Clocks Used	Global Clocks Avail	FF Used	FF Avail	LUTM Used	LUTM Avail	RAMB36 Used
X0Y0	0	12	0	2600	0	600	0
X1Y0	0	12	0	1500	0	550	0
X0Y1	1	12	23	2000	0	600	0
X1Y1	0	12	0	1900	0	650	0
X0Y2	0	12	0	2000	0	600	0
X1Y2	1	12	516	1900	147	650	0
X0Y3	0	12	0	2600	0	600	0
X1Y3	0	12	0	1350	0	500	0

5. Device Cell Placement — Global Clock g0

Global Id	Driver Type/Pin	Clock	Period (ns)	Slice Loads	IO Loads	GT Loads
g0	BUFG/O	—	—	516	0	0

6. Device Cell Placement — Global Clock g1

Global Id	Driver Type/Pin	Clock	Period (ns)	Slice Loads	IO Loads	GT Loads
g1	BUFG/O	sys_clk_pin	10.000	23	0	0

7. Clock Region Cell Placement — Region X0Y1 (g1)

Global Id	Track	Driver Type/Pin	Clock Loads	Non-Clock Loads	FF	Memory LUTs
g1	n/a	BUFG/O	23	0	23	0

8. Clock Region Cell Placement — Region X1Y2 (g0)

Global Id	Track	Driver Type/Pin	Clock Loads	Non-Clock Loads	FF	Memory LUTs
g0	n/a	BUFG/O	516	0	516	0

Part 3 — Power Report

1. Summary

Parameter	Value
Total On-Chip Power (W)	0.145
Design Power Budget (W)	Unspecified
Dynamic (W)	0.048
Device Static (W)	0.097
Effective TJA (C/W)	4.6
Max Ambient (C)	84.3
Junction Temperature (C)	25.7
Confidence Level	Low

1.1 On-Chip Components

On-Chip	Power (W)	Used	Available	Utilization (%)
Clocks	<0.001	3	—	—
Slice Logic	0.010	1347	—	—
LUT as Logic	0.009	590	63,400	0.93
Register	<0.001	539	126,800	0.43
LUT as Distributed RAM	<0.001	32	19,000	0.17
F7/F8 Muxes	<0.001	97	63,400	0.15
CARRY4	<0.001	39	15,850	0.25
Others	0.000	10	—	—

On-Chip	Power (W)	Used	Available	Utilization (%)
BUFG	0.000	1	32	3.13
Signals	0.015	1188	—	—
I/O	0.022	15	210	7.14
Static Power	0.097	—	—	—
Total	0.145	—	—	—

1.2 Power Supply Summary

Source	Voltage (V)	Total (A)	Dynamic (A)	Static (A)	Budget (A)	Margin (A)
Vccint	1.000	0.041	0.026	0.015	Unspecified	NA
Vccaux	1.800	0.019	0.001	0.018	Unspecified	NA
Vcco33	3.300	0.010	0.006	0.004	Unspecified	NA
Vcco25	2.500	0.000	0.000	0.000	Unspecified	NA
Vcco18	1.800	0.000	0.000	0.000	Unspecified	NA
Vcco15	1.500	0.000	0.000	0.000	Unspecified	NA
Vcco135	1.350	0.000	0.000	0.000	Unspecified	NA
Vcco12	1.200	0.000	0.000	0.000	Unspecified	NA
Vccaux_io	1.800	0.000	0.000	0.000	Unspecified	NA
Vccbram	1.000	0.000	0.000	0.000	Unspecified	NA
MGTA Vcc	1.000	0.000	0.000	0.000	Unspecified	NA
MGTA Vtt	1.200	0.000	0.000	0.000	Unspecified	NA
Vccadc	1.800	0.020	0.000	0.020	Unspecified	NA

2.2 Clock Constraints

Clock	Domain	Constraint (ns)
sys_clk_pin	CLK100MHZ	10.0

3.1 Power by Hierarchy

Name	Power (W)
top	0.048
cpu	0.025
dp	0.025
pcreg	0.007
rf	0.017

7.7 Before-and-After Functional Comparison

Metric	Initial (Buggy)	Final (Fixed)
EDA Playground Simulation	FAILED — waveform not generated	PASSED — all waveforms correct
Vivado Simulation	FAILED — elaboration errors	PASSED — simulation runs clean
FPGA Synthesis	FAILED — \$readmemh, port errors	PASSED — bitstream generated
FPGA Hardware Deployment	NOT POSSIBLE — no board I/O	PASSED — Arty A7-100T deployed
ALU Operations	All 4 ops: WRONG results	ADD/SUB/AND/OR all correct
Branch Instructions	Branch target WRONG (wrong SrcA)	Branch target correct (PC+8+offset)
Conditional Execution	CondEx X-propagation → all X	All 16 ARM conditions functional
Register File Reset	All registers X at startup	All regs cleared to 0 on reset
Instruction Memory	X if memfile.dat not found	All 23 instructions pre-loaded
Test Result (R5==11)	FAIL — clock never toggled	PASS — R5 correctly = 32'd11
Vivado DRC Warnings	Multiple critical warnings	Zero DRC errors or warnings
FPGA Debug Output	None	LED: PC/ALUResult/Instr/WData

Table 7.3 — Comprehensive before-and-after functional comparison

8. Novelty and Contribution

This project makes several distinct contributions to the educational and practical literature on ARM processor HDL implementation and FPGA deployment.

8.1 Systematic Multi-Layer Bug Analysis

The primary contribution is the comprehensive, layer-by-layer diagnosis and correction of a real-world buggy Verilog implementation. The 15 identified bugs span every tier of the design hierarchy, from fundamental Verilog language misuse (output reg on wire-driven ports, casex hazards, SystemVerilog keywords in Verilog files) through microarchitectural errors (swapped ALU op-codes, wrong branch SrcA) to tool-specific issues (\$readmemh incompatibility with Vivado synthesis, include chain conflicts). No single existing resource documents all these error categories together in the context of a unified ARM processor design.

8.2 Complete FPGA Deployment Infrastructure

The initial code was simulation-only with no FPGA deployment path. This project contributes a complete FPGA infrastructure layer consisting of:

- A 21-bit hardware debounce circuit implementing a 20ms stability window at 100MHz - a practical design pattern applicable to any button-input FPGA design
- A rising-edge step-clock generator enabling human-observable single-instruction execution on physical hardware
- A 2-bit multiplexed LED debug interface providing real-time visibility of four independent 32-bit CPU signals
- A complete Arty A7-100T XDC constraint file with all required port-to-pin mappings and IOSTANDARD declarations

This infrastructure transforms a simulation-only academic design into a fully interactive, step-debuggable hardware system.

8.3 Hardcoded ROM Strategy for FPGA-Portable Programs

The replacement of \$readmemh file-based memory initialisation with an inline Verilog initial block containing hardcoded instructions is a practically significant technique. This approach guarantees that Vivado synthesises the instruction ROM as a fully pre-initialised Block RAM or distributed ROM, eliminating the file-path dependency that makes \$readmemh unreliable in synthesis flows. The NOP-fill strategy (filling unused ROM locations with `MOV R0,R0 = 0xE1A00000`) provides a safe, predictable response to PC overrun. This pattern is directly applicable to any future FPGA-targeted Verilog processor design.

8.4 Full 16-Condition condcheck Module

The absence of the condcheck module in the initial code, combined with the incomplete condition code set that the module represented, is corrected by the implementation of a complete 16-condition evaluator with the correctly derived GE signal ($ge = \sim(N \wedge V)$ per ARM architecture specification). This enables the processor to execute the full range of ARM conditional instructions, including the infrequently implemented LT, GT, LE, and HI conditions, which are exercised by the test program.

8.5 Cross-Tool Verilog Compatibility

The corrected design compiles and simulates correctly in both EDA Playground (Icarus Verilog, Verilog-2001 strict mode) and Xilinx Vivado (synthesis and behavioural simulation). This cross-tool compatibility was achieved by eliminating all SystemVerilog-only constructs (input logic, output logic keywords), standardising sensitivity list syntax (comma $\rightarrow$ or keyword), adding consistent timescale 1ns/1ps directives across all modules, and removing include chains in favour of project-level module resolution. This portability is a direct and replicable contribution for practitioners using multiple EDA toolchains.

8.6 Educational Debugging Reference

The systematic documentation of each bug - its symptom, root cause, propagation effect, and specific fix - provides a detailed reference for students and practising engineers encountering similar issues in Verilog HDL projects. The categorisation of bugs into five recurring error classes (signal type misuse, X-propagation, op-code mapping errors, tool compatibility issues, and missing FPGA infrastructure) provides a structured debugging methodology that is generally applicable beyond this specific design.

9. Conclusion

This project successfully achieved all stated objectives. A multi-cycle ARM-subset processor sourced from GitHub in a non-functional state, containing 15 critical bugs across 10 Verilog modules, was systematically diagnosed, corrected, simulated, synthesised, and deployed on a Digilent Arty A7-100T FPGA board.

The most impactful correction was the ALU operation code remapping, which resolved the fundamental arithmetic incorrectness that rendered the processor entirely non-functional. The branch SrcA fix (RegSrc = 2'b01 for branch instructions) restored correct PC-relative addressing. The elimination of output reg misuse across datapath.v, condlogic.v, controller.v, and dmem.v resolved all multi-driver synthesis errors. The replacement of \$readmemh with a hardcoded initial block made the design FPGA-synthesisable. The addition of the condcheck module with all 16 ARM condition codes enabled full conditional execution capability.

The FPGA deployment infrastructure - comprising the 21-bit debounce circuit, step-clock generator, LED multiplexer, and XDC constraint file - transformed the design from a simulation exercise into an interactive, physically observable hardware system. The step-by-step execution mode, where each push of btn[1] advances the processor by exactly one instruction while the LEDs display selectable CPU state, proved highly effective for in-hardware verification of the 23-instruction test program.

The final design achieved:

- Zero X-propagation in simulation from cycle 0
- All testbench assertions passing (R5 = 32'd11)
- Zero Vivado DRC errors or critical warnings
- Successful FPGA bitstream generation and board deployment
- Correct execution of all 19 active instructions including 3 branches (2 taken, 1 skipped) and 1 conditional instruction
- Full observability of CPU state via the multiplexed LED debug interface

The work demonstrates that systematic debugging methodology, careful attention to Verilog signal type rules, and purpose-built FPGA infrastructure are the three essential ingredients for successfully transitioning an academic processor design from simulation to working silicon.

10. Future Scope

The completed single-cycle ARM processor provides a solid, verified foundation for several natural extensions. The following directions are proposed for future development.

10.1 Pipelined Processor Architecture

The current design is single-cycle: one instruction completes per clock cycle, with the critical path determining the maximum clock frequency. A five-stage pipeline (IF → ID → EX → MEM → WB) would increase throughput dramatically but requires hazard detection and forwarding logic to handle read-after-write data hazards and control hazards from branch instructions. The verified single-cycle datapath provides the ideal reference for validating each pipeline stage in isolation using the existing testbench infrastructure.

10.2 Extended Instruction Set

The current processor supports ADD, SUB, AND, ORR, LDR, STR, and branch instructions. The ARM Thumb-2 instruction set additionally includes MUL (multiply), MLA (multiply-accumulate), LSL/LSR/ASR (shift operations), BL (branch with link for function calls), and multi-register transfer instructions (LDM/STM). Extending the decoder and ALU to support these instructions would significantly broaden the range of programs that can be executed.

10.3 UART Serial Interface for Program Loading

The current processor requires programs to be hardcoded into `imem.v` at synthesis time. Integrating a UART receiver with the Arty A7-100T's USB-UART bridge would enable programs to be loaded at runtime from a host computer without requiring Vivado re-synthesis. This would make the board function as a general-purpose embedded processor development platform.

10.4 Seven-Segment Display Interface

The current LED debug interface provides only 4 bits of display resolution. Adding a seven-segment display decoder (or connecting an external Pmod 7-segment module) would allow full 8-digit hexadecimal display of 32-bit register values, making program verification substantially more convenient without requiring an oscilloscope or logic analyser.

10.5 Cache Memory Hierarchy

The current design uses combinational ROM for instruction memory and a single-cycle synchronous RAM for data memory. A realistic memory hierarchy with separate L1 instruction and data caches (backed by the 256MB DDR3L on the Arty A7-100T via the MIG memory controller) would introduce cache miss handling, memory stalls, and cache coherence considerations — advancing the design towards industry-representative complexity.

10.6 Formal Verification

The current verification relies on simulation testbenches. Formal property verification using tools such as Symbiyosys or Cadence JasperGold would provide mathematical proof of correctness properties such as "the ALU Carry flag is always equal to NOT borrow for subtraction" or "RegWrite is never asserted during a branch instruction." Formal verification would catch corner-case bugs that simulation may miss, particularly in the conditional execution logic.

10.7 Interrupt and Exception Handling

The current processor has no interrupt mechanism. Adding an interrupt controller, exception vector table, and supervisor mode would enable the processor to handle timer interrupts, software exceptions, and undefined instruction traps - bridging the gap between this educational design and production-quality embedded processors.

11. References

The following references were consulted in the design, debugging, and documentation of this project:

[Ref]	Citation
[1]	Harris, D. M. and Harris, S. L. (2021). Digital Design and Computer Architecture: ARM Edition. 2nd ed. Morgan Kaufmann. ISBN: 978-0128000564. [Primary architectural reference for the processor module hierarchy, instruction encoding, and datapath design.]
[2]	Patterson, D. A. and Hennessy, J. L. (2013). Computer Organisation and Design: ARM Edition. Morgan Kaufmann. ISBN: 978-0128017333. [Foundational reference for RISC processor microarchitecture, single-cycle and pipelined datapaths.]
[3]	Digilent Inc. (2023). Arty A7 Reference Manual. Available at: https://digilent.com/reference/programmable-logic/arty-a7/reference-manual . [Board-level reference for Arty A7-100T pin assignments, I/O standards, clock sources, and peripheral connections.]
[4]	Xilinx / AMD (2023). Vivado Design Suite User Guide: RTL Analysis and HDL Elaboration (UG901). Available at: https://docs.xilinx.com/r/en-US/ug901-vivado-design-analysis . [Tool reference for synthesis, elaboration, and DRC error resolution in Vivado.]
[5]	Xilinx / AMD (2023). Vivado Design Suite User Guide: Synthesis (UG901). Available at: https://docs.xilinx.com/r/en-US/ug901-vivado-synthesis . [Reference for synthesis constraints, initial block handling, and Block RAM inference.]
[6]	ARM Limited (2021). ARM Architecture Reference Manual, ARMv7-A and ARMv7-R Edition. Available at: https://developer.arm.com/documentation/ddi0406/latest . [Definitive specification for ARM condition codes, flag definitions, and instruction encoding.]
[7]	IEEE (2001). IEEE Standard for Verilog Hardware Description Language. IEEE Std 1364-2001. [Language standard reference for sensitivity list syntax, port type rules, and simulation/synthesis semantics.]
[8]	Sutherland, S., Davidmann, S., and Flake, P. (2006). SystemVerilog for Design. 2nd ed. Springer. [Reference for distinguishing Verilog-2001 and SystemVerilog constructs — particularly the logic keyword and always_comb restrictions.]
[9]	Kuon, I. and Rose, J. (2007). Measuring the gap between FPGAs and ASICs. IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems, 26(2), pp. 203–215. [Reference for FPGA resource utilisation metrics and area/performance characterisation.]
[10]	Trimberger, S. M. (2018). Three Ages of FPGAs: A Retrospective on the First Thirty Years of FPGA Technology. Proceedings of the IEEE, 106(3), pp. 610–616. [Survey of FPGA technology evolution and modern device capabilities.]
[11]	EDA Playground (2024). Online HDL Simulation Platform. Available at: https://edaplayground.com . [Platform used for initial simulation verification of the corrected design in Icarus Verilog / Verilog-2001 mode.]
[12]	Ciletti, M. D. (2010). Advanced Digital Design with the Verilog HDL. 2nd ed. Prentice Hall. [Reference for parameterised module design, flip-flop coding styles, and memory modelling best practices in Verilog.]
[13]	CoreEL Technologies (I) Pvt. Ltd. (2024). Arty A7-100T: Artix-7 FPGA Development Board Datasheet. Part Number 410-319-1. Bangalore. [Provided hardware specifications: FPGA part

XC7A100TCSG324-1, 101,440 logic cells, 240 DSP slices, 4,860 Kbits BRAM, DDR3L, clock sources.]

Table 11.1 — Full reference list